Assessing the Accuracy of a One Dimensional Heat Equation Model for Heat Flow in a Rod

Michael Taleff

January 9, 2020

1 Parameters and Assumptions of the Problem

In this demonstration, a rod initially held at room temperature (20°C) had its end put in contact with liquid nitrogen (77 Kelvin). The change in temperature with time was measured at positions halfway and a fourth of the way along the rod.

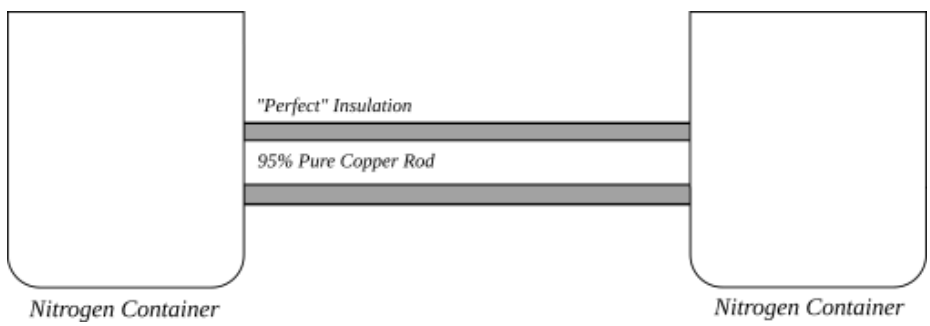

Figure 1: Diagram of the experimental setup where the copper rod was insulted along its length and welded to two metal cups at its ends. These cups contained liquid nitrogen throughout the experiment.

We may assume the initial temperature distribution of the rod was uniform at room temperature. For the purposes of this model, I will also assume perfect heat transfer between the liquid nitrogen and the copper rod along with a constant liquid nitrogen temperature. Finally, I will also assume the copper rod is completely homogeneous and that the insulation surrounding the rod is perfect. According to the materials database on therm-test.com, the diffusivity of the copper is about $69 * 10^{-6}$ m^2/s.

2 Derivation of Single Dimension Transient Heat Flow

2.1 Separation of Variables

In order to model the heat flow in the rod, we use the one dimension heat equation, whose solutions give a description of the evolution of a spatial distribution of heat.

$$\frac{\partial}{\partial t}T(x,t) = \alpha \frac{\partial^2}{\partial x^2}T(x,y)$$

In order to make this partial differential equation easier to solve, we make it homogeneous by substituting $T(x,t)$ with $\Theta(x,t)$ where $T(x,t) = \Theta(x,t) + T_0$. To solve this partial differential equation, we assume that $\Theta(x,t)$ is separable (i.e. it may be written as $T(x,t) = \Gamma(t)\Psi(x)$).

$$\frac{\partial}{\partial t}(\Gamma(t)\Psi(x)) = \alpha \frac{\partial^2}{\partial x^2}(\Gamma(t)\Psi(x))$$

Since $\Psi(x)$ does not depend on t, we may move it out of the partial since it is treated as a constant. The same may be done for $\Gamma(t)$ with respect to x.

$$\frac{\partial}{\partial t}(\Gamma(t)) * \Psi(x) = \alpha \frac{\partial^2}{\partial x^2}(\Psi(x)) * \Gamma(t)$$

$$\alpha^{-1}\frac{\partial}{\partial t}(\Gamma(t)) * \Gamma^{-1}(t) = \frac{\partial^2}{\partial x^2}(\Psi(x)) * \Psi^{-1}(x)$$

Since both sides of the above equation depend on different variables, they must both equate to a constant. (e.g., if the left side was equal to some function of t, then the right side would have to depend on t, which is not possible since $\Psi(x)$ is only a function of x). Let that constant be $-\lambda^2$.

$$\alpha^{-1}\frac{\partial}{\partial t}(\Gamma(t)) * \Gamma^{-1}(t) = -\lambda^2 = \frac{\partial^2}{\partial x^2}(\Psi(x)) * \Psi^{-1}(x)$$

This leaves us with two ordinary differential equations that we may solve separately.

2.2 Solving the Spatial ODE

$$\frac{\partial^2}{\partial x^2}(\Psi(x)) * \Psi^{-1}(t) = -\lambda^2$$

$$\frac{\partial^2}{\partial x^2}\Psi(x) = -\lambda^2\Psi(x)$$

After we solve this ordinary differential equation, we can get the following equation for $\Psi(x)$.

$$\Psi(x) = A_1\cos(\lambda x) + A_2\sin(\lambda x)$$

Since $\Theta(x,t)$ is homogeneous at the boundaries of our rod, $\Theta(x,t) = 0$ at the edges, which means $\Psi(x) = 0$ at $x = 0$ and $x = L$. If we plug in these boundary equations, we get the following equations.

$$0 = A_1\cos(0) + A_2\sin(0) = A_1$$
$$0 = A_1\cos(\lambda L) + A_2\sin(\lambda L)$$

Since A_1 is zero, we can simplify our second boundary condition equation.

$$0 = A_2\sin(\lambda L) \Rightarrow n\pi = \lambda L \text{ where } n \in \mathbb{Z}^+$$
$$\lambda = \frac{n\pi}{L} \text{ where } \mathbb{Z}^+$$

This means our solution to our spatial ODE is:

$$\Psi(x) = A_2\sin(\frac{n\pi}{L}x)$$

2.3 Solving the Temporal ODE

$$\alpha^{-1}\frac{\partial}{\partial t}(\Gamma(t)) * \Gamma^{-1}(t) = -\lambda^2$$
$$\frac{\partial}{\partial t}\Gamma(t) = -\alpha\lambda^2\Gamma(t)$$

After we solve this ordinary differential equation, we derive the following relation for $\Gamma(t)$ and plug in the relation we found for λ earlier.

$$\Gamma(t) = \exp(-\alpha(\frac{n\pi}{L})^2 t)$$

2.4 Constructing a General Solution

Using our equations for $\Gamma(t)$ and $\Psi(x)$, we may find a general solution for $T(x,t) = \Theta(x,t) + T_0$. A solution for $\Theta(x,t)$ is equal to $\Gamma(t)\Psi(x)$.

$$\Theta(x,t) = A_n \sin(\frac{n\pi}{L}x)\exp(-\alpha(\frac{n\pi}{L})^2 t)$$

For a homogeneous differential equation, any linear combination of solutions is itself a solution. In order to fulfill our initial condition, we can take a linear combination of solutions based off our above general solution to find the actual solution. Luckily, this general solution is periodic, so we may invoke Fourier's Theorem to guarantee the existence of a solution that matches our initial temperature distribution (we can drop the exponential term since $\exp(0) = 1$).

$$F(x) - T_0 = \sum_{n=1}^{\infty} A_n \sin(\frac{n\pi}{L}x)$$

In order to find the coefficients of our Fourier series A_n, we may use a trick involving the following integral:

$$\int_0^L \sin(\frac{n\pi}{L}x)\sin(\frac{m\pi}{L}x)dx = \begin{cases} \frac{L}{2} \text{ if } m = n \\ 0 \text{ if } m \neq n \end{cases}$$

If we multiple both sides of our equation by $\sin(n\pi x/L)$ and take the integral, we get the following series:

$$\begin{aligned} \int_0^L (F(x) - T_0)\sin(\frac{n\pi}{L}x)dx = &\int_0^L A_1 \sin(\frac{\pi}{L}x)\sin(\frac{n\pi}{L}x)dx \\ &+ \int_0^L A_2 \sin(\frac{2\pi}{L}x)\sin(\frac{n\pi}{L}x)dx + \\ &+ \int_0^L A_3 \sin(\frac{3\pi}{L}x)\sin(\frac{n\pi}{L}x)dx + ... \end{aligned}$$

Using the earlier integral we defined, any term where the sine functions do not have the same argument drop out, leaving the following relation.

$$\int_0^L (F(x) - T_0)\sin(\frac{n\pi}{L}x)dx = \int_0^L A_n \sin(\frac{n\pi}{L}x)\sin(\frac{n\pi}{L}x)dx$$

We can also evaluate the right side of the equation to be $L/2$, giving the final relation for the coefficients of our Fourier series, A_n.

$$\frac{2}{L}\int_0^L (F(x) - T_0)\sin(\frac{n\pi}{L}x)dx = A_n$$

Evaluating this integral for our initial temperature distribution, $F(x) = T_i$, gives our final expression for A_n.

$$A_n = \begin{cases} (T_i - T_0)\frac{4}{n\pi} & \text{if } n \text{ is odd} \\ 0 & \text{if } n \text{ is even} \end{cases}$$

This gives us our theoretical temperature with respect to time, where α is the diffusivity of the copper.

$$T(x,t) = T_0 + \sum_{n=1,3,5,...}^{\infty} \frac{4(T_i - T_0)}{n\pi}\sin(\frac{n\pi}{L}x)\exp(-\alpha(\frac{n\pi}{L})^2 t)$$

3 Results

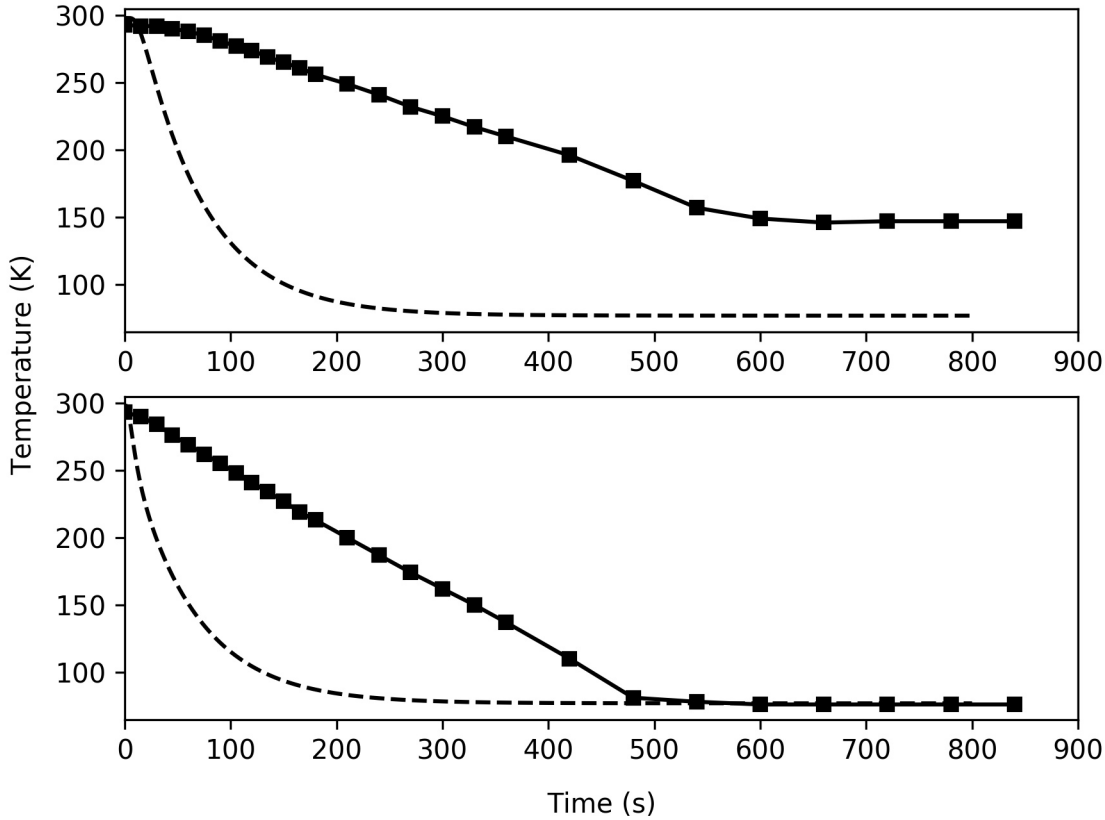

Figure 2: The top graph is the temperature at the center of the rod over time, with the dashed curve representing the theoretical temperature curve and the solid line graphing the actual temperature change. Each dot represents a data point. The bottom curve shows the temperature change one fourth of the way along the copper rod.

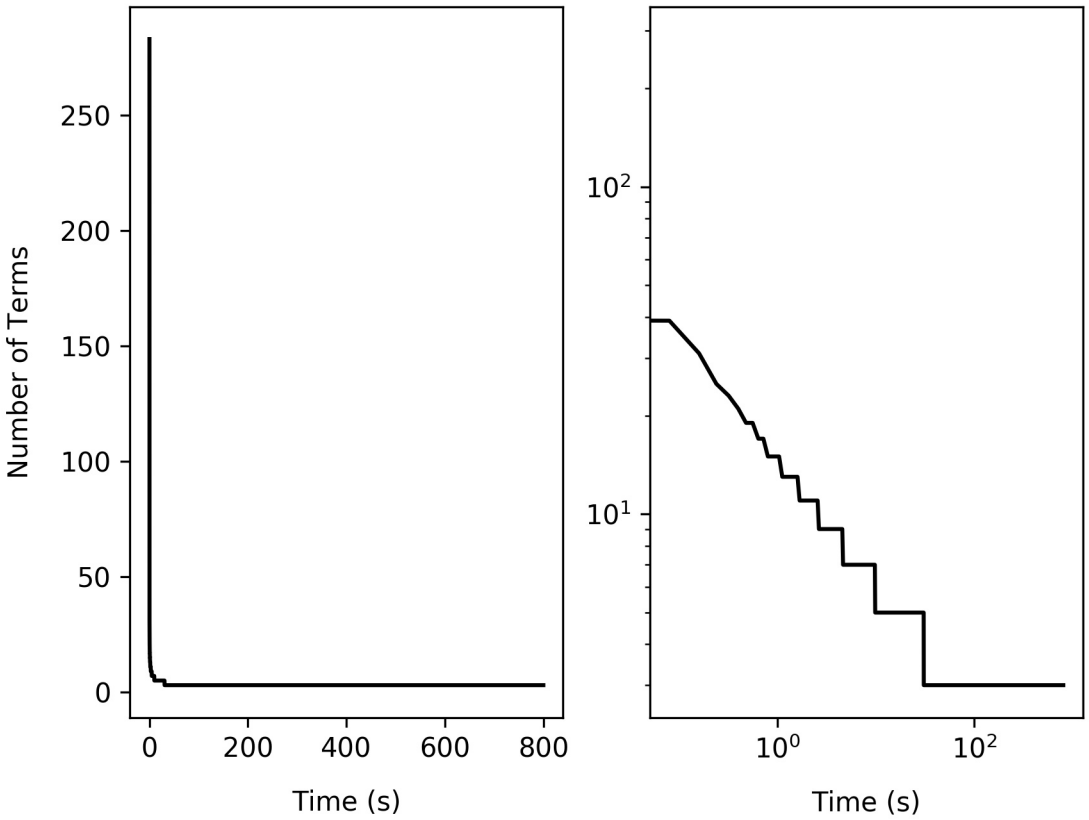

Figure 3: The left graph shows the number of terms required in the Fourier series over time. The right displays the same data but with logarithmic axes.

The Fourier series and number of terms needed were calculated using a python script. As shown by the chart, there is a large difference between the actual temperature and the theoretical temperature. This could be due to a number of issues with the assumptions made. One problematic assumption is the assumption of an instantaneous heat transfer between the copper rod and the liquid nitrogen. This assumption would be less problematic if the rate of heat transfer in copper was slow compared to the rate of heat transfer between copper and liquid nitrogen. However, the rate of heat transfer in copper is very fast and as the nitrogen warms up, it boils, creating air pockets between the copper rod and the liquid nitrogen, decreasing the effective heat transfer coefficient between the rod and the liquid nitrogen. This calculation also assumed perfect insulation, which is not physically possible.

4 Calculation Code

4.1 Fourier Series

```python
import numpy as np


# Function that returns the nth element of a fourier series
def fourier_element(x_value, coefficient=0, n=1, length=1,
                    diffusivity=1, time=0, insulated=False):

    # fourier_lambda is the argument of the trig functions
    # in a fourier expansion
    fourier_lambda = (n*np.pi) / length

    # If the bar is insulated on the ends, the cosine fourier
    # expansion is used, otherwise the sine expansion is used
    if insulated:
        return coefficient * np.cos(fourier_lambda * x_value) \
               * np.exp(-diffusivity*(fourier_lambda**2)*time)
    return coefficient * np.sin(fourier_lambda * x_value) \
           * np.exp(-diffusivity*(fourier_lambda**2)*time)


# Evaluates a fourier expansion at a certain point
# The coefficients parameter must be a function that returns
# the value of the nth coefficient in the fourier series
def fourier_series(x_value, terms, constant, coefficients,
                   length=1, diffusivity=1, time=0, insulated=False):

    value = constant
    for term in range(1, terms+1):
        coefficient = coefficients(term)
        value += fourier_element(x_value, coefficient,
                                 term, length, diffusivity, time,
                                 insulated)

    return value
```

4.2 Rod Model

```
import numpy as np
import Fourier

# Experimental Values
# Thermocouple one is the thermocouple located at L/2
# Thermocouple two is the thermocouple located at L/4
exper_times = (0, 15, 30, 45, 60, 75, 90, 105, 120, 135, 150,
               165, 180, 210, 240, 270, 300, 330, 360, 420, 480,
               540, 600, 660, 720, 780, 840)
exper_thermocouple_one = (293.15, 292.15, 292.15, 290.15, 288.15,
                          285.15, 281.15, 277.15, 274.15, 269.15,
                          265.15, 261.15, 256.15, 249.15, 241.15,
                          232.15, 225.15, 217.15, 210.15, 196.15,
                          177.15, 157.15, 149.15, 146.15, 147.15,
                          147.15, 147.15)
exper_thermocouple_two = (293.15, 290.15, 284.15, 276.15, 269.15,
                          262.15, 255.15, 248.15, 241.15, 234.15,
                          227.15, 219.15, 213.15, 200.15, 187.15,
                          174.15, 162.15, 150.15, 137.15, 110.15,
                          81.15, 78.15, 76.15, 76.15, 76.15,
                          76.15, 76.15)

# List of all values needed for the model
diffusivity = 69 * 10**(-6)
english_length = 8  # inches
metric_length = english_length / 39.37  # meters
liquid_nitrogen_temp = 77  # Kelvin
initial_temp = 298.15  # Kelvin
thermocouple_one = 4  # inches
metric_thermocouple_one = thermocouple_one / 39.37   # meters
thermocouple_two = 2  # inches
metric_thermocouple_two = thermocouple_two / 39.37  # meters
start_time = 0  # seconds
end_time = 800  # seconds


# Coefficients of the fourier series for a flat line
def coefficients(n):
    if n % 2:
        return (initial_temp-liquid_nitrogen_temp) * (4/(n*np.pi))
    return 0


# Generation of a series of time points for temperature calculation
samples = 10000  # This is a totally arbitrary number
times = np.linspace(start_time, end_time, samples)

# Calculations for the number of terms needed
# Since every even term is zero, we must check all the
# odd numbered terms, hence the incrementation by two
# The value of the next odd term must be less than one
# to guarantee the series is off by less than
```

```python
# This fourier series is alternating, so the deviation
# is always less than the absolute value of the next term
kept_terms_list = []
for time in times:
    term_number = 1
    next_term = 2  # Placeholder
    while abs(next_term) > 1:
        term_number += 2
        next_term = Fourier.fourier_element(metric_thermocouple_one,
                                            coefficients(term_number),
                                            term_number, metric_length,
                                            diffusivity, time)
    kept_terms_list.append(term_number)

# Take the max of the kept_term-list to guarantee the whole
# model is less than 1 Kelvin off at any time
n_val = max(kept_terms_list)

# Calculations for the first thermocouple
thermocouple_one_temps = Fourier.fourier_series(metric_thermocouple_one,
                                                n_val, liquid_nitrogen_temp,
                                                coefficients, metric_length,
                                                diffusivity, times)

# Calculations for the second thermocouple
thermocouple_two_temps = Fourier.fourier_series(metric_thermocouple_two,
                                                n_val, liquid_nitrogen_temp,
                                                coefficients, metric_length,
                                                diffusivity, times)
```

4.3 Plots

```
import matplotlib.pyplot as plt
import RodModel

# ACS Recommendations: no plot title, no color

# Creation of a figure with two vertically stacked subplots
fig, thermocouples = plt.subplots(2, 1, sharey='all')

# Adding an invisible frame to the plot to create a shared
# y-axis label
frame = fig.add_subplot(111, frameon=False)
frame.tick_params(labelcolor='none', top=False, bottom=False,
                  left=False, right=False)

# Plotting the experimental and theoretical temperature plots for
# the copper rod
thermocouples[0].plot(RodModel.exper_times,
                      RodModel.exper_thermocouple_one,
                      'sk-', markersize=5)
thermocouples[0].plot(RodModel.times,
                      RodModel.thermocouple_one_temps, 'k--')
thermocouples[0].set_xlim(0, 900)
thermocouples[0].set_ylim(65, 305)

thermocouples[1].plot(RodModel.exper_times,
                      RodModel.exper_thermocouple_two,
                      'sk-', markersize=5)
thermocouples[1].plot(RodModel.times,
                      RodModel.thermocouple_two_temps, 'k--')
thermocouples[1].set_xlim(0, 900)
thermocouples[1].set_ylim(65, 305)
thermocouples[1].set_xlabel('Time (s)', labelpad=10)
plt.ylabel('Temperature (K)', labelpad=10)

fig.savefig('figure_experimental_versus_theoretical.jpg', dpi=300)

fig_two = plt.figure()
terms_plot = fig_two.add_subplot(121)
terms_plot.plot(RodModel.times, RodModel.kept_terms_list, 'k-')
terms_plot.set_xlabel('Time (s)', labelpad=10)
terms_plot.set_ylabel('Number of Terms', labelpad=10)

terms_plot = fig_two.add_subplot(122)
terms_plot.loglog(RodModel.times, RodModel.kept_terms_list, 'k-')
terms_plot.set_xlabel('Time (s)', labelpad=10)

fig_two.savefig('figure_number_of_terms.jpg', dpi=300)
```